



Seminario de Actualización · Tecnicatura Superior en Ciencia de Datos e IA · IFTS N.º 18

SEMINARIO DE ACTUALIZACIÓN · CLASE 6

Consumir un modelo de IA por API

Vocabulario de Python · pipeline e InferenceClient · revisión de pares

```
$ pip install huggingface_hub
>>> client = InferenceClient(...)
>>> client.text_classification(
...     "se entiende bien")
Positive 0.8
$ git clone <repo del compañero>
```

DOCENTE

Cynthia M. Villagra

CICLO LECTIVO

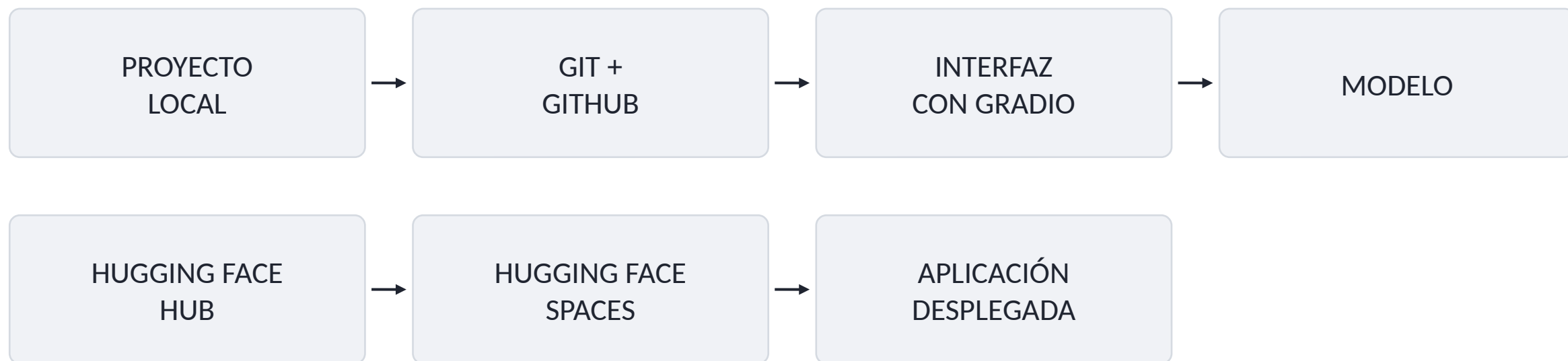
2026 · Segundo cuatrimestre

FECHA

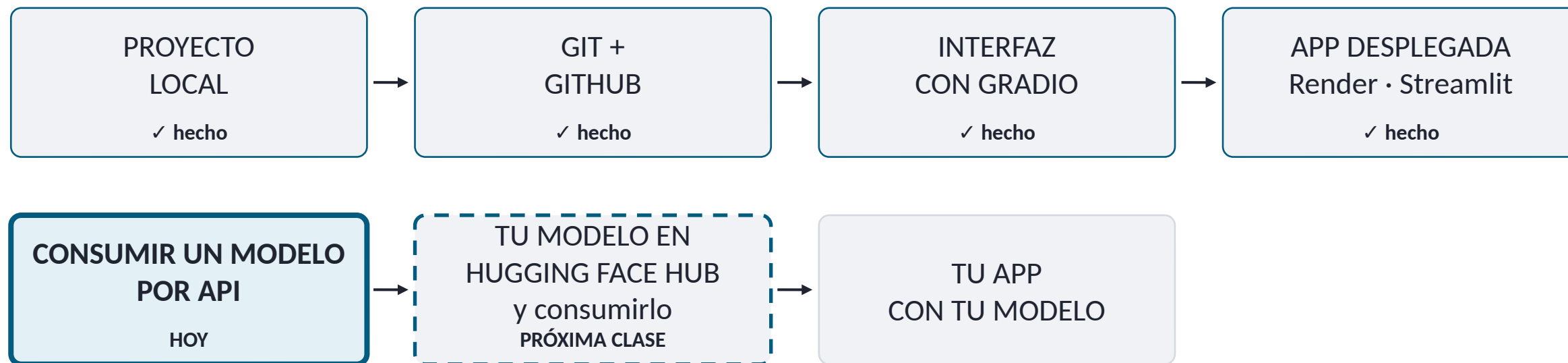
Viernes 25/09/2026

La ruta que nos propusimos en la Clase 1

El recorrido completo del seminario, tal como lo planteamos al empezar.



La ruta, como la venimos haciendo



Cambió el orden, no el destino: una app desplegada que usa tu propio modelo.

Con qué te vas hoy

1. El **vocabulario** para leer código ajeno: función, parámetro, retorno, clase, objeto, atributo, método.
2. Tu **primer llamado a un modelo de IA** funcionando, desde tu terminal.
3. Saber **dónde corre el modelo** y por qué eso cambia lo que podés hacer.
4. Un **diagnóstico de tu portfolio** hecho por un compañero que no te pudo preguntar nada.
5. La actividad de la semana: el modelo sumado a **tus dos apps**, en Render y en Streamlit.

Del 1 al 5: ¿alguien puede correr tu repo sin preguntarte?

En la planilla está tu **nombre**, tu **repo** y el **número del 1 al 5** que pusiste.

Planilla de repos (también en el aula virtual)

https://docs.google.com/spreadsheets/d/1hKxXlzDMuuxvZwFiMk98g6FI6iDKYY0xM_l0vKFE0_s/edit?usp=sharing

Lo vamos a comprobar al final de la clase.

¿Cómo lo programarías?

Hoy tu app de Gradio recibe un texto y hace exactamente lo que vos programaste — por ejemplo, saludar.

Querés que, cuando alguien escribe un comentario, la app te diga si es **positivo**, **negativo** o **neutro**.

¿Cómo lo programarías?

¿Y con estas frases?

"No está nada mal"

"Buenísimo, otra vez se cayó el servidor"

Si la regla es "si dice *bueno* es positivo, si dice *malo* es negativo"... ¿qué pasa con estas?

Las reglas escritas a mano no alcanzan: hace falta un modelo de IA ya entrenado.

No vamos a programar todo nosotros

Vamos a usar una librería de Python que escribió Hugging Face:

```
huggingface_hub
```

Trae sus propias piezas, por ejemplo `InferenceClient` y `text_classification`.

Para entender qué es cada una, primero necesitamos ponerles nombre.

Estas líneas las escribiste vos

```
def saludar(nombre):  
    return f"Hola, {nombre}"  
  
demo = gr.Blocks()  
entrada = gr.Textbox(label="Escribí algo")  
boton.click(saludar, inputs=entrada, outputs=salida)  
demo.launch(server_name="0.0.0.0")
```

- ¿Qué es saludar? ¿Y nombre?
- ¿Qué es demo? ¿Y label?
- ¿Qué hace el punto en boton.click?

Una función y una clase, lado a lado

Función

```
def saludar(nombre):           # nombre: parámetro
    return f"Hola, {nombre}"   # lo que devuelve

mensaje = saludar("Ana")      # "Ana": argumento
# mensaje → "Hola, Ana"
```

La función recibe **PARÁMETROS** y **RETORNA** un valor.

Clase

```
class Cliente:
    def __init__(self, nombre):
        self.nombre = nombre      # atributo

    def saludar(self):             # método
        return f"Hola, {self.nombre}"

    def cambiar_nombre(self, nuevo): # método
        self.nombre = nuevo

c = Cliente("Ana")               # instanciar → objeto
c.nombre                         # atributo → "Ana"
c.saludar()                      # método → "Hola, Ana"
```

La clase se **INSTANCIA**: recibe argumentos entre paréntesis y **DEVUELVE UN OBJETO**, con **atributos** y **métodos**.

Vocabulario de funciones

Término	Qué es	En tu código
Función	recibe parámetros, hace algo y devuelve un resultado	<code>saludar(nombre)</code>
Parámetro	el nombre que aparece en la definición	<code>nombre</code> en <code>def saludar(nombre)</code>
Argumento	el valor que le pasás al llamarla	<code>"Ana"</code> en <code>saludar("Ana")</code>
Valor de retorno	lo que la función devuelve con <code>return</code>	<code>"Hola, Ana"</code>

Vocabulario de clases

Término	Qué es	En tu código
Clase	un molde que agrupa datos y comportamiento	<code>gr.Textbox, gr.Blocks, Cliente</code>
Instanciar	crear el objeto llamando a la clase	<code>gr.Textbox(...), Cliente("Ana")</code>
Objeto / instancia	lo que se crea a partir de la clase	<code>entrada, demo, c</code>
Atributo	un dato que guarda el objeto	<code>entrada.label, c.nombre</code>
Método	una función que pertenece al objeto	<code>demo.launch(), c.saludar()</code>

¿Qué es PEP 8?

PEP = *Python Enhancement Proposal*: los documentos oficiales donde la comunidad de Python propone y acuerda cambios y convenciones.

PEP 8 es la guía de estilo oficial del código Python.

Es una **convención**: Python no la obliga y el código corre igual. Pero casi todas las librerías la respetan, así que el nombre te da una pista.

Qué es	Estilo	Ejemplo
Clase	CapWords	InferenceClient, Textbox
Función / método	snake_case	text_classification, launch
Variable	snake_case	entrada, client
Constante	MAYÚSCULAS	HF_TOKEN

peps.python.org/pep-0008 → "Naming Conventions"

Convención ≠ regla: el nombre es una pista, no una prueba.

Cómo verlo en VS Code, sin memorizar

C en el autocompletado → clase
Tooltip: `class ...`

f en el autocompletado → función o método
Tooltip: `(function) ...`

Cuestionario de repaso

Repaso de lo que acabamos de ver. **No lleva nota.**

Link en el chat.

[link al cuestionario de repaso]

¿Qué es pipeline y para qué se usa?

`pipeline` es de la librería `transformers`, de Hugging Face.

- Es la forma más simple de **usar un modelo ya entrenado** para una tarea: clasificar texto, traducir, resumir...
- Esconde el código complejo: prepara el texto, se lo pasa al modelo y ordena la respuesta.
- Se usa sobre todo para **probar un modelo rápido**, en tu compu o en Colab.

¿pipeline es una clase o una función?

```
from transformers import pipeline

pipe = pipeline(
    "text-classification",
    model="cardiffnlp/twitter-xlm-roberta-base-sentiment",
)
pipe("Esta clase está buenísima")
```

A · pipeline es una **clase**

B · pipeline es una **función**

¿En qué te basaste?

SOLUCIÓN

Le preguntamos a Python

```
>>> type(pipeline)
<class 'function'>
```

`type()` no adivina por el nombre: te dice qué es de verdad.

Es una **función**: la pista de PEP 8 era correcta.

Pero si es una función... ¿qué quedó guardado en pipe?

Si es una función... ¿qué quedó guardado en pipe?

```
pipe = pipeline("text-classification", model="...")
```

Un texto

Un número

Una lista

Otra cosa

Lo que recibiste es un objeto

```
>>> type(pipe)
<class 'transformers.pipelines.text_classification.
      TextClassificationPipeline'>
```

TextClassificationPipeline está en CapWords → es una **clase**.
Entonces pipe es un **objeto** (una instancia) de esa clase.

Llamaste a una función. Te devolvió un objeto.

¿Qué forma tiene lo que devuelve pipe?

```
>>> pipe("Esta clase está buenísima")  
[{'label': 'Positive', 'score': 0.9...}]
```

Guardá esta respuesta: la vamos a necesitar más adelante.

pipeline por dentro

Recortado del código real de `transformers` (la función tiene unas 450 líneas):

```
def pipeline(task=None, model=None, ...):  
    ...  
    # 1. según la tarea, elige qué clase usar  
    pipeline_class = targeted_task["impl"]  
    #     "text-classification" → TextClassificationPipeline  
    ...  
    # 2. crea el objeto de esa clase y lo devuelve  
    return pipeline_class(model=model, task=task, **kwargs)
```

Es una función: por dentro instancia una clase y te devuelve el objeto.

Una función puede devolver un objeto

El nombre de la función te dice qué **llamás**.
`type ( )` te dice qué **recibiste**.

Puente: `client.text_classification(...)` es un método. ¿Qué te va a devolver?

Entrenar e inferir

Entrenar

Proceso en el que se ajustan los parámetros (pesos) de un modelo a partir de un conjunto de datos, para que aprenda a resolver una tarea. Requiere muchos datos y mucho cómputo.

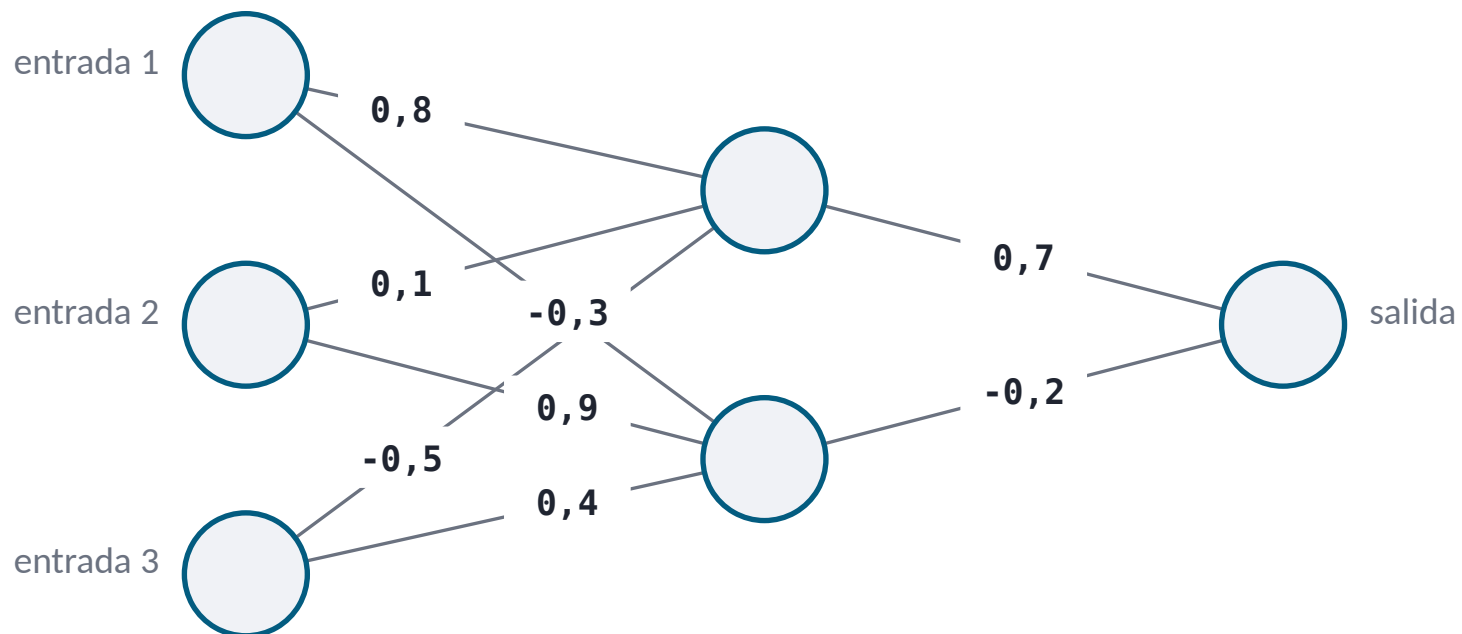
En este modelo lo hizo Cardiff NLP.

Inferir (inferencia)

Usar un modelo ya entrenado, con sus pesos fijos, para obtener una predicción sobre datos nuevos. No modifica el modelo.

Es lo que hacemos hoy.

¿Qué es un peso?



Los números sobre las líneas son **pesos**: cuánto influye cada entrada en la neurona siguiente.

Entrenar = ajustar esos números.
Inferir = usarlos fijos.

Este modelo tiene **cientos de millones** de pesos: por eso el archivo ocupa 1,11 GB.

¿Dónde corrió el modelo?

Cuando usamos `pipeline` en Colab, ¿en qué computadora se hizo la inferencia?

¿Podés usar pipeline en tu app de Render?

¿Por qué sí? ¿Por qué no?

Sí, porque...

No, porque...

No es porque sea local: es porque no entra

El modelo pesa
1,11 GB
+ PyTorch cargado en memoria

Render gratis te da
512 MB de memoria

- **Se puede instalar:** Render también es una computadora que corre Python.
- **No alcanza la memoria** del plan gratis. Con un plan pago, sí: es una cuestión de costo.

O pagás una máquina que lo aguante, o se lo pedís a otra: InferenceClient.

pipeline vs. InferenceClient: dónde corre el modelo

	pipeline	InferenceClient
Dónde corre el modelo	en tu máquina (o en Colab)	en los servidores del proveedor
Qué instalás	<code>transformers</code> + PyTorch, y bajás el modelo	<code>huggingface_hub</code> , liviano
Qué necesitás	RAM y disco para el modelo	internet y un token
Quién pone el límite	tu hardware	el proveedor: cuota y disponibilidad
Tus datos	no salen de tu máquina	viajan al proveedor

Con la API el límite lo pone el proveedor; con pipeline, tu hardware.

¿Qué es una model card?

Es el README .md del repositorio del modelo: la información para poder usarlo.

Metadatos (YAML, arriba)

Tarea, idiomas, licencia, datasets. Sirven para que el Hub filtre y muestre el modelo.

Texto para personas

Qué hace el modelo y cómo usarlo.

Debería contar: el modelo, **para qué está pensado y sus limitaciones**, cómo se entrenó, **con qué datos y cómo se evaluó**.

Abrí la model card y encontrá tres cosas

```
huggingface.co/cardiffnlp/twitter-xlm-roberta-base-sentiment
```

1. ¿Qué **etiquetas** devuelve?
2. ¿En qué **idiomas** se entrenó?
3. ¿Con qué **datos**?

3 minutos

Lo que dice la model card

- **Etiquetas:** `Positive`, `Neutral`, `Negative` — en inglés y con mayúscula inicial. Copiá la etiqueta exacta, no la traducción.
- **Idiomas:** ajustado en 8 — árabe, inglés, francés, alemán, hindi, italiano, español y portugués.
- **Datos:** entrenado con ~198 millones de tweets y ajustado para sentimiento.
- **Extra:** el ejemplo reemplaza `@usuario` por `@user` y los links por `http`. Espera texto de Twitter.

AHORA PENSÁ

¿Qué le falta a esta model card?

La documentación dice que debería contar **para qué está pensado** el modelo, **sus limitaciones** y **cómo se evaluó**.

¿Lo encontraron en esta?

La card es el primer lugar donde mirar, pero no siempre está completa.

¿Qué frases va a clasificar mal?

Este modelo aprendió leyendo **tweets**: textos cortos e informales.

Pensá en frases que una persona entiende bien, pero que leídas palabra por palabra podrían confundir.

¿Qué es una API, en una frase?

- Tu código **manda un pedido** a otra computadora y **recibe una respuesta**.
- El **token** sirve para probar quién sos.
- Cada llamado **tarda algo**, y la cuenta gratis tiene **límites de uso**.

CONSIGNA

Paso 1 • El token en la terminal

PowerShell

```
$env:HF_TOKEN = "hf_..."
```

Git Bash

```
export HF_TOKEN="hf_..."
```

OJO

La sintaxis es distinta. La de Git Bash, pegada en PowerShell, falla.

Paso 2 • ¿Qué forma va a tener la respuesta?

```
import os
from huggingface_hub import InferenceClient

client = InferenceClient(
    provider="hf-inference",
    api_key=os.environ["HF_TOKEN"],
)

resultado = client.text_classification(
    "esta clase me está costando pero se entiende",
    model="cardiffnlp/twitter-xlm-roberta-base-sentiment",
    top_k=3,
)

for r in resultado:
    print(r.label, r.score)
```

¿Un texto?
¿Un número?
¿Una lista?

Leelo con el vocabulario

En el código	Qué es
<code>InferenceClient</code>	la clase
<code>client</code>	el objeto, la instancia
<code>provider, api_key</code>	parámetros; lo que va después del <code>=</code> , los argumentos
<code>text_classification</code>	un método de <code>client</code>
<code>top_k=3</code>	pide las 3 etiquetas más probables (doc oficial)
<code>resultado</code>	valor de retorno: <code>list[TextClassificationOutputElement]</code> , una lista de objetos
<code>r.label, r.score</code>	atributos de cada elemento

¿Quién dio este error: tu compu o Hugging Face?

401 Unauthorized

Token mal copiado.
Lo responde **Hugging Face**: el pedido sí llegó.

KeyError: 'HF_TOKEN'

La variable no está definida.
Lo da **tu compu**, antes de mandar nada.

pipeline vs. InferenceClient: en el código

	<code>pipeline</code>	<code>InferenceClient</code>
Qué es	función que devuelve un objeto	clase que instanciás
El modelo se elige	al crear el pipeline	en cada llamado al método
Qué devuelve	lista de diccionarios: <code>r["label"]</code>	lista de objetos: <code>r.label</code>
Cuántas etiquetas	por defecto, solo la ganadora	las que pidas con <code>top_k</code>
Token	<code>token=</code>	<code>api_key=</code>

Si copiás un tutorial de una y lo pegás en la otra, se rompe.

AHORA PENSÁ

Probá con una frase irónica

Volvé a lo que anotamos en la model card.

¿El modelo la entendió como la entendiste vos?

Sumá el análisis de sentimiento a tus dos apps

1. **Gradio en Render:** una pestaña nueva (gr . Tab), sin tocar lo que ya andaba. HF__TOKEN en *Environment*.
2. **Streamlit en Community Cloud:** el mismo llamado. HF__TOKEN en *Advanced settings* → *Secrets*.
3. `huggingface__hub` en el `requirements . txt` de las dos.
4. En `Clase_06/README . md`: los dos links y dónde configuraste el token (**sin el token**).
5. **Entrega:** el link a tu repositorio.

Si no llegás con todo: primero Render, después Streamlit.

AHORA PENSÁ

La única diferencia real en Streamlit

```
@st.cache_resource
def crear_cliente():
    return InferenceClient(provider="hf-inference",
                           api_key=os.environ["HF_TOKEN"])
```

Streamlit vuelve a correr todo el script en cada clic.
¿Qué pasaría con el objeto `cliente` sin la primera línea?

Ahora, el repo de un compañero

- Buscá en la planilla el repo del compañero que te tocó.
- Clonalo y revisalo **sin hablar con el autor**. Solo lectura: `git clone`, nunca push.
- Antes de clonar: escribí en una línea **qué esperás encontrar**. No lo compartas todavía.

Planilla de repos: https://docs.google.com/spreadsheets/d/1hKxXlzDMuuxvZwFiMk98g6FI6iDKYYQxM_l0vKFEQ_s/edit?usp=sharing

Si el repo es privado, está vacío o no existe: pedí el repo banco. • Clonalo cerca de la raíz (`C:\repos\`).

La rúbrica

Punto	Sí	Más o menos	No
El README general explica de qué es el proyecto sin preguntar			
Hay una carpeta por clase (02 a 05) y se entiende cuál es cuál			
Cada carpeta alcanza para reproducir esa clase			
Pudiste correr o probar algo sin trabarte			
El historial de commits muestra el progreso			

Ticket de salida

En el foro "**Ticket de salida — Clase 6**":

1. el link del repo que revisaste
2. la rúbrica completada
3. **un problema concreto** que contraste (con el mensaje de error, si hubo)

Si no llegás en clase: hasta el domingo 27/09, 23:59.

AHORA PENSÁ

¿Qué faltaba de verdad?

Al principio de la clase pusiste en la planilla un número del 1 al 5. Después de revisar el repo de otro: **¿qué faltaba o no funcionaba de verdad? ¿Coincide con el número que pusiste?**

[glosario de lo que se rompió, armado en vivo]

Documentar es para el que no te puede preguntar

Documentar no es prolijidad: es la diferencia entre que otro pueda usar lo que hiciste o no.

Empezaron usando el modelo de otro y terminan leyendo el repo de otro.

Las dos cosas piden lo mismo: entender código que no escribiste.

Antes de irte

- **Ticket de salida:** en el foro, hoy o hasta el domingo 27/09, 23:59.
- **Actividad:** hasta el viernes 2/10, 23:59. Primero Render, después Streamlit.

La clase que viene: subís tu propio modelo a Hugging Face y lo consumís.